

Code Layout, Readability, and Reusability

CrediShield AI — Credit Card Approval Prediction System

1. Project Directory Structure

The repository is organized into a modular structure separating data, models, backend logic, frontend assets, and documentation.

Credit-Card-Approval-Prediction/	
├─ data/	# Raw datasets (excluded from Git via .gitignore)
│ └─ application_record.csv	# Applicant demographics (438,510 rows)
│ └─ credit_record.csv	# Payment history (1,048,575 rows)
├─ models/	# Serialized ML artifacts (loaded at runtime)
│ └─ card_model.joblib	# Trained GradientBoostingClassifier
│ └─ scaler.joblib	# Fitted StandardScaler
│ └─ label_encoders.joblib	# Dictionary of 11 fitted LabelEncoders
│ └─ feature_names.joblib	# Ordered feature name list (18 features)
│ └─ metrics.json	# Model comparison scores
│ └─ stats.json	# Training dataset statistics
├─ templates/	
│ └─ index.html	# Jinja2 template – single-page dashboard structure
├─ static/	
│ └─ style.css	# Glassmorphism theme, animations, layout styling
│ └─ script.js	# Client logic, fetch API calls, Chart.js rendering
├─ Project Documentation/	# Comprehensive phase-by-phase documentation PDFs
│ └─ 1. Brainstorming & Ideation/	
│ └─ 2. Requirement Analysis/	
│ └─ 3. Project Design Phase/	
│ └─ 4. Project Planning Phase/	
│ └─ 5. Project Development Phase/	
│ └─ 6. Project Testing/	
├─ train.py	# Automated ML pipeline script
├─ app.py	# Flask backend server & inference engine
├─ notebook.ipynb	# Jupyter notebook for interactive EDA
├─ requirements.txt	# Production dependencies (pinned versions)
├─ requirements-train.txt	# Training/development dependencies
├─ Procfile	# Render deployment start command (`web: gunicorn app:app`)
├─ .python-version	# Python runtime definition (3.12)
├─ .gitignore	# Git tracking exclusion rules
└─ README.md	# Setup instructions and project overview

2. Code Readability Standards Applied

The codebase adheres to best practices to ensure it is understandable, maintainable, and beginner-friendly.

Readability Standard	Implementation in Project
Inline Comments	Complex logic is explained. Example in app.py: # Compute engineered features or # Scale numerical features (passing DataFrame to suppress warnings)
Section Separators	Long files use banner comments (# =====) to delineate distinct functional modules.
Docstrings	Functions and Flask routes include triple-quoted docstrings describing inputs, outputs, and purpose.
Meaningful Variable Naming	Variables describe their contents exactly (e.g., income_per_member, encoded_data, scaled_num_features, prob_approved) instead of ambiguous names like x, y, temp.
Constants in UPPERCASE	File paths and settings are defined at the module level (e.g., MODEL_PATH = 'models/card_model.joblib').
Descriptive Error Messages	Try-catch blocks return clear JSON errors pointing to the specific issue (e.g., "Missing input fields: gender" or "Failed to load metrics:...").
Consistent Formatting	4-space indentation for Python files; 2-space indentation for HTML/CSS/JS.

3. Reusability and Modularity Patterns

The system is designed with decoupled components that can be upgraded or reused independently.

3.1 Dynamic Feature Alignment

In app.py, the incoming data is not hardcoded into an array. Instead, it dynamically reads the feature_names.joblib file generated during training:

<pre>features = [] for col in feature_names: if col in encoded_data: features.append(encoded_data[col]) elif col in scaled_num_features: features.append(scaled_num_features[col])</pre>
--

Benefit: If new features are added to the ML model in the future, the prediction endpoint automatically adjusts to the correct input shape and order without requiring code rewrites.

3.2 Externalized Preprocessors

Categorical encoders and numerical scalers are fitted in `train.py` and saved as `.joblib` files. `app.py` simply loads and applies them.

Benefit: Ensures that the exact transformation logic used during training is identically applied during production inference, preventing data leakage or distribution shifts.

3.3 Metrics Caching

Evaluation scores (Accuracy, F1) and dataset statistics (demographic distributions) are calculated once in `train.py` and saved as `metrics.json` and `stats.json`. The `/metrics` API endpoint simply reads these files.

Benefit: The web dashboard can render complex analytical charts instantly without needing to hold the full dataset in memory or recalculate statistics on the fly.

3.4 Self-Healing Startup (Auto-Training)

The Flask app checks for required model files upon startup. If they are missing (e.g., after a fresh clone of the repo), it dynamically imports and executes the training pipeline:

```
if not os.path.exists(MODEL_PATH):
    print("Model not found! Initializing train.py pipeline...")
    from train import run_training_pipeline
    run_training_pipeline()
```

Benefit: The application is self-contained. A new developer doesn't need to manually run prerequisite scripts; simply running `python app.py` sets up the entire environment.

3.5 CSS Custom Properties (Theme System)

In `style.css`, all colors, borders, and spacing values are defined as CSS variables attached to the `:root` and `[data-theme="light"]` selectors.

```
:root {
  --bg-primary: #0a0f1e;
  --text-primary: #f8fafc;
  /* ... */
}
```

Benefit: Implementing a dark/light mode toggle required changing just ~20 variables rather than maintaining two entirely separate stylesheets.

3.6 Dependency Segregation

Dependencies are split into `requirements.txt` (production) and `requirements-train.txt` (development).

Benefit: Keeps the production deployment lightweight (~150MB instead of ~800MB), circumventing cloud provider bundle size limits while preserving heavy visualization tools (Matplotlib, Seaborn) for local ML development.

Project Name: CrediShield AI
Author: Dinesh (dineshTech-creator)
Date: July 2026